

Sri Lanka Institute of Information Technology



Assignment Part 2. A React Frontend Application
Application Frameworks
(SE3040)

SE_26

AGRIHUB-LK – Backend Testing & Quality
Assurance Report

1. Introduction

This report outlines the comprehensive testing strategy implemented for the **AgriHUB-LK** backend. The objective of this testing phase was to ensure the reliability, security, and scalability of the core logistics engine. The project utilizes a full-stack MERN architecture (MongoDB, Express, React, Node.js), and this report focuses specifically on the backend quality assurance pipeline.

1.1 Scope of Testing

- **Unit Testing:** Validation of individual controller logic in isolation using mocked dependencies.
 - **Integration Testing:** Validation of end-to-end API endpoints, middleware, and database interactions using a mock environment.
 - **Performance Testing:** Stress-testing the system under simulated user loads to determine breaking points and latency.
-

2. Testing Environment & Tools

The following technologies were utilized to establish a robust QA pipeline:

Tool	Purpose
Jest	Primary test runner and assertion library.
Supertest	HTTP assertions for testing RESTful API routes.
MongoDB Memory Server	Isolated, high-speed database environment for integration tests.
Artillery	Performance and load testing tool for stress simulation.
Babel	Transpiling ES6 modules for compatibility with the testing suite.

3. Unit Testing (Controller Level)

Unit tests were designed to verify the business logic of controllers (e.g., tripController.js, vehicleController.js) by mocking dependencies like database models and external services.

3.1 Methodology

- **Isolation:** Extensive use of jest.mock() to prevent actual database calls and focus on logic.
 - **Request/Response Mocking:** Utilizing node-mocks-http to simulate Express req and res objects.
 - **Scenarios:** Coverage includes "Happy Path" (successful execution) and "Edge Cases" (validation errors, unauthorized access, and invalid inputs).
-

4. Integration Testing (Route Level)

Integration tests verified the interaction between the Express routes, the Auth middleware, and the MongoDB models.

4.1 Key Test Suites

- **Vehicle Management:** Verified that only **Transporters** can create or update vehicle records, with strict validation for fuelType, capacity, and vehicleType enums.
- **Trip Management:** Tested the automated workflow where creating a trip updates the vehicle status to On Delivery and links it to the corresponding Distributor's order.

4.2 Security & RBAC

Tests confirmed that **Role-Based Access Control (RBAC)** correctly blocks unauthorized roles. For example, a Farmer is blocked from accessing Transporter-specific trip statistics or vehicle management routes.

5. Performance & Load Testing

Performance testing was conducted using **Artillery** to ensure the system can handle the high traffic expected during peak harvest seasons in Sri Lanka.

5.1 Test Configuration (load-test.yml)

- **Warm-up Phase:** 5 virtual users/second for 60 seconds to prime the server.
- **Ramp-up Phase:** Gradual increase to 50 virtual users/second over 2 minutes.
- **Peak Load:** Sustained 50 users for a duration of 2 minutes to monitor stability.

5.2 Data-Driven Testing (test-data.csv)

Utilized a CSV dataset to simulate diverse user logins and unique order IDs. This prevents "caching skew" and ensures the database is queried realistically.

6. Test Execution & Results

All tests were executed within the backend environment using pre-configured NPM scripts.

6.1 Execution & Replication Guide

To verify the testing suite, follow this standardized procedure:

Step 1: Environment Setup

Navigate to the backend directory and install all necessary dependencies:

Bash

```
cd backend  
npm install
```

Step 2: Execution Commands

Run the specific test suites using the following commands:

- **Unit Tests:** `npm run test:unit`
- **Integration Tests:** `npm run test:int`
- **Performance Tests:** `artillery run performance/load-test.yml`
- **Full Comprehensive Report:** `npm run test:all`

6.2 Coverage & Performance Summary

The system achieved a high percentage of code coverage and met all performance benchmarks:

- **Controllers:** > 90% Line Coverage.
- **Routes:** 100% Endpoint connectivity and middleware validation.
- **Performance:** Average response time remained under **200ms** during the Peak Load phase.

7. Conclusion

The backend of **AgriHUB-LK** demonstrates high resilience and stability. By implementing a multi-layered testing strategy, we have ensured that the system is not only functionally correct but also secure against unauthorized access and capable of handling production-level traffic. The use of an in-memory database for integration tests ensures that the test suite is fast, repeatable, and does not pollute production data.